



SE3082 - Parallel Computing

Assignment – 02

3rd year 1st semester - 2026

IT23575608 – H. W. Ranwala

Contents

1. ENVIRONMENT SETUP AND TUTORIAL	3
1.1 OPENMP INSTALLATION AND SETUP	4
1.2 MPI INSTALLATION (OPENMPI)	4
1.3 CUDA INSTALLATION AND SETUP (GOOGLE COLAB)	4
1.4 TROUBLESHOOTING & COMMON ISSUES	6
2. PROGRAM IMPLEMENTATION AND COMPILATION.....	8
2.1 BEFORE OPTIMIZATION	8
1. <i>OpenMP</i>	8
2. <i>MPI</i>	10
3. <i>CUDA</i>	12
2.2 AFTER OPTIMIZATION	15
1. <i>OpenMP</i>	15
2. <i>MPI</i>	17
3. <i>CUDA</i>	20

1. Environment Setup and Tutorial

System Overview

This assignment utilizes a hybrid environment: local execution for CPU-based parallel frameworks (OpenMP, MPI) and a cloud platform for GPU-based execution (CUDA).

Component	Specification	Justification for Parallel Computing
Operating System	Ubuntu 22.04.4 LTS (64-bit)	Native support for Linux development tools and stable library management.
CPU	AMD Ryzen 7 5700U	Features 8 Physical Cores and 16 Threads, ideal for high-scale OpenMP and MPI testing.
GPU (Cloud)	NVIDIA Tesla T4 (Google Colab)	Provides 2,560 CUDA Cores and 16GB GDDR6 memory, surpassing local entry-level GPUs for performance benchmarking.
RAM	8GB DDR4	Sufficient for managing parallel process overhead and large vector/matrix datasets.

Environment Justification

- AMD Ryzen 7 5700U:** With 16 logical threads, this processor allows for significant speedup demonstrations when scaling OpenMP thread counts.
- Google Colab (T4 GPU):** Since CUDA requires specific NVIDIA hardware, utilizing a T4 instance ensures "High-Performance Computing" (HPC) grade testing without local hardware limitations.
- Ubuntu 22.04:** This OS is the industry standard for MPI and OpenMP, offering the most straightforward installation of libopenmpi-dev and gcc.

Prerequisites Installation (Local Ubuntu)

Before setting up the frameworks, update the system and install essential build tools:

```
sudo apt update && sudo apt upgrade -y
sudo apt install build-essential wget curl git -y
```

Note: build-essential includes the gcc and g++ compilers required for all frameworks.

1.1 OpenMP Installation and Setup

OpenMP is integrated directly into the GNU Compiler Collection (GCC). No separate installation is required but ensure the latest GCC is present.

- **Installation:**

`sudo apt install gcc -y`

- **Verification:**

`gcc --version`

```
Execution time: 0.000000 seconds
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There
is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PU
RPOSE.

heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$
```

- **Compilation:** Use the `-fopenmp` flag to link the OpenMP library.

`gcc -fopenmp vector_add.c -o vector_add_omp`

1.2 MPI Installation (OpenMPI)

MPI (Message Passing Interface) is used for distributed memory parallelism.

- **Installation:**

`sudo apt install openmpi-bin libopenmpi-dev -y`

- **Verification:**

`mpirun --version`

```
Execution time: 0.103924 seconds
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$ mpirun --version
mpirun (Open MPI) 4.1.2

Report bugs to http://www.open-mpi.org/community/help/
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$
```

- **Compilation:** Use the `mpicc` wrapper for MPI C programs.

`mpicc vector_add_mpi.c -o vector_add_mpi`

- **Execution:** Run with `-np` (number of processes). For your Ryzen 7, you can test up to 16.

`mpirun -np 8 ./vector_add_mpi`

1.3 CUDA Installation and Setup (Google Colab)

Because we are using Google Colab for CUDA, the drivers and toolkit are pre-installed, but hardware verification is required for the report.

- **Step 1:** In Colab, go to Runtime > Change runtime type and select T4 GPU.

- **Verification (Add this to your report logs):**

`!nvidia-smi`

```
!nvidia-smi

Sun May 10 10:17:54 2026

+-----+
| NVIDIA-SMI 580.82.07                Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU  Name           Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|   0   Tesla T4               Off  | 00000000:00:04.0 Off  |          0          |
| N/A   51C    P8             16W /  70W |  0MiB / 15360MiB |      0%      Default |
+-----+-----+-----+-----+-----+-----+

Processes:
+-----+-----+-----+-----+-----+-----+
| GPU  GI   CI           PID  Type  Process name                      GPU Memory |
|   ID  ID   ID                                   Usage   |
+-----+-----+-----+-----+-----+-----+
| No running processes found |
+-----+-----+-----+-----+-----+-----+

```

`!nvcc --version`

```
!nvcc --version

nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Fri_Feb_21_20:23:50_PST_2025
Cuda compilation tools, release 12.8, V12.8.93
Build cuda_12.8.r12.8/compiler.35583870_0

```

- **Compilation:** Use the NVIDIA Cuda Compiler (nvcc).

`!nvcc vector_add_cuda.cu -o vector_add_cuda`

- **Execution:**

`!./vector_add_cuda`

1.4 Troubleshooting & Common Issues

Note: These steps are reproducible for any 3rd-year student using a local Ubuntu 22.04 environment or Google Colab.

Issue 1: NVCC Command Not Found (Google Colab)

- **Cause:** The NVIDIA CUDA Compiler (`nvcc`) is not in the system path, or the GPU runtime is not enabled.
- **Fix:**
 - 1. First, ensure the GPU accelerator is active: Runtime > Change runtime type > GPU.
 - 2. If the error persists in a cell, manually map the path: `bash import os`
`os.environ['PATH'] += '/usr/local/cuda/bin'`
 - 3. Verify by running `!nvcc --version`.

Issue 2: nvidia-smi Fails to Initialize (Local vs. Colab)

- **Cause:** On your local Ubuntu (Ryzen 7), this will fail because you do not have an NVIDIA GPU. On Colab, it fails if the driver isn't loaded.
- **Fix:**
 - **Local:** This is expected. Use the CPU-based fallback (OpenMP/MPI) for local work.
 - **Colab:** Restart the runtime (Runtime > Restart session) to re-initialize the NVIDIA driver.

Issue 3: MPI Library/Header Not Found

- **Cause:** The compiler cannot find `mpi.h` or the `mpicc` wrapper.
- **Fix:**
 - Install the development headers on your local Ubuntu:
`sudo apt update`
`sudo apt install openmpi-bin libopenmpi-dev -y`

Issue 4: OpenMP Code Runs Sequentially (No Parallelism)

- **Cause:** The program was compiled without the `-fopenmp` flag, or the environment variable for threads isn't set.
- **Fix:**
 - Ensure you compile with the flag: `gcc -fopenmp vector_add.c -o v_add`.
 - Set the thread count to match your Ryzen 7 (16 threads) before running:

```
export OMP_NUM_THREADS=16
./v_add
```

Issue 5: Permission Denied for Execution

- **Cause:** The compiled binary does not have execution permissions.
- **Fix:**
 - Apply the execute permission using `chmod`:

```
chmod +x vector_add_mpi
./vector_add_mpi
```

2. Program Implementation and Compilation

2.1 Before Optimization

1. OpenMP

```
openmp > c vector_add_openmp.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4  #include <sys/time.h>
5
6  #define N 10000000 // Array size
7
8  // Helper function to get current time in seconds
9  double get_time() {
10     struct timeval tv;
11     gettimeofday(&tv, NULL);
12     return tv.tv_sec + tv.tv_usec * 1e-6;
13 }
14
15 int main() {
16     int *A, *B, *C;
17     double start_time, end_time;
18
19     // Allocate memory for the arrays
20     A = (int*)malloc(N * sizeof(int));
21     B = (int*)malloc(N * sizeof(int));
22     C = (int*)malloc(N * sizeof(int));
23
24     if (A == NULL || B == NULL || C == NULL) {
25         printf("Memory allocation failed!\n");
26         return 1;
27     }
28
29     // Initialize arrays
30     for (int i = 0; i < N; i++) {
31         A[i] = i;
32         B[i] = i * 2;
33     }
34
35     start_time = get_time();
36
37     // Perform vector addition using OpenMP
38     #pragma omp parallel for
39     for (int i = 0; i < N; i++) {
40         C[i] = A[i] + B[i];
41     }
42
43     end_time = get_time();
44
45     // Verify results (check first and last elements)
46     printf("OpenMP Vector Addition\n");
47     printf("Array size: %d\n", N);
48     printf("C[0] = %d (Expected: %d)\n", C[0], 0 + 0);
49     printf("C[%d] = %d (Expected: %d)\n", N-1, C[N-1], (N-1) + (N-1)*2);
50     printf("Execution time: %f seconds\n", end_time - start_time);
51
52     // Free memory
53     free(A);
54     free(B);
55     free(C);
56
57     return 0;
58 }
59
```

```
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$ gcc -fopenmp openmp
/vector_add_openmp.c -o openmp/vector_add_openmp
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$ ./openmp/vector_add
_openmp
OpenMP Vector Addition
Array size: 10000000
C[0] = 0 (Expected: 0)
C[9999999] = 29999997 (Expected: 29999997)
Execution time: 0.008886 seconds
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$
```

2. MPI

```
mpi > c vector_add_mpi.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  #define N 10000000 // Total array size
6
7  int main(int argc, char** argv) {
8      int rank, size;
9      int *A = NULL, *B = NULL, *C = NULL;
10     int *local_A, *local_B, *local_C;
11     double start_time, end_time;
12
13     // Initialize MPI environment
14     MPI_Init(&argc, &argv);
15     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
16     MPI_Comm_size(MPI_COMM_WORLD, &size);
17
18     // Ensure array size is divisible by number of processes
19     // (For a beginner tutorial, this keeps the code simpler)
20     if (N % size != 0) {
21         if (rank == 0) {
22             printf("Error: Array size %d must be divisible by number of processes %d.\n", N, size);
23         }
24         MPI_Finalize();
25         return 1;
26     }
27
28     int local_N = N / size;
29
30     // Root process initializes the full arrays
31     if (rank == 0) {
32         A = (int*)malloc(N * sizeof(int));
33         B = (int*)malloc(N * sizeof(int));
34         C = (int*)malloc(N * sizeof(int));
35
36         if (A == NULL || B == NULL || C == NULL) {
37             printf("Memory allocation failed!\n");
38             MPI_Abort(MPI_COMM_WORLD, 1);
39         }
40
41         for (int i = 0; i < N; i++) {
42             A[i] = i;
43             B[i] = i * 2;
44         }
45     }
46 }
```

```

46
47 // Allocate memory for local arrays on all processes
48 local_A = (int*)malloc(local_N * sizeof(int));
49 local_B = (int*)malloc(local_N * sizeof(int));
50 local_C = (int*)malloc(local_N * sizeof(int));
51
52 // Synchronize processes before starting the timer
53 MPI_Barrier(MPI_COMM_WORLD);
54 if (rank == 0) start_time = MPI_Wtime();
55
56 // Scatter the arrays A and B from root to all processes
57 MPI_Scatter(A, local_N, MPI_INT, local_A, local_N, MPI_INT, 0, MPI_COMM_WORLD);
58 MPI_Scatter(B, local_N, MPI_INT, local_B, local_N, MPI_INT, 0, MPI_COMM_WORLD);
59
60 // Perform local vector addition
61 for (int i = 0; i < local_N; i++) {
62     local_C[i] = local_A[i] + local_B[i];
63 }
64
65 // Gather the local results back to the root process array C
66 MPI_Gather(local_C, local_N, MPI_INT, C, local_N, MPI_INT, 0, MPI_COMM_WORLD);
67
68 // Synchronize again to accurately measure time
69 MPI_Barrier(MPI_COMM_WORLD);
70
71 if (rank == 0) {
72     end_time = MPI_Wtime();
73     printf("MPI Vector Addition\n");
74     printf("Array size: %d, Processes: %d\n", N, size);
75     printf("C[0] = %d (Expected: %d)\n", C[0], 0 + 0);
76     printf("C[%d] = %d (Expected: %d)\n", N-1, C[N-1], (N-1) + (N-1)*2);
77     printf("Execution time: %f seconds\n", end_time - start_time);
78
79     free(A);
80     free(B);
81     free(C);
82 }
83
84 free(local_A);
85 free(local_B);
86 free(local_C);
87
88 // Finalize MPI environment
89 MPI_Finalize();
90 return 0;
91 }
92

```

```

heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$ mpicc mpi/vector_add_mpi.c -o mpi/vector_add_mpi
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$ mpirun -np 4 ./mpi/vector_add_mpi
MPI Vector Addition
Array size: 10000000, Processes: 4
C[0] = 0 (Expected: 0)
C[9999999] = 29999997 (Expected: 29999997)
Execution time: 0.105924 seconds
heshan@heshan-Swift3:~/Desktop/PC---Assignment-2$

```

3. CUDA

```
cuda > vector_add_cuda.cu
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <cuda_runtime.h>
4
5  #define N 10000000 // Array size
6
7  // CUDA Kernel function for vector addition
8  // __global__ indicates this runs on the GPU and is called from the host (CPU)
9  __global__ void vectorAdd(const int *A, const int *B, int *C, int numElements) {
10     // Calculate global thread ID
11     int i = blockDim.x * blockIdx.x + threadIdx.x;
12
13     // Ensure we don't access out of bounds if threads > array size
14     if (i < numElements) {
15         C[i] = A[i] + B[i];
16     }
17 }
18
19 int main(void) {
20     size_t size = N * sizeof(int);
21
22     // Allocate host (CPU) memory
23     int *h_A = (int *)malloc(size);
24     int *h_B = (int *)malloc(size);
25     int *h_C = (int *)malloc(size);
26
27     if (h_A == NULL || h_B == NULL || h_C == NULL) {
28         printf("Failed to allocate host vectors!\n");
29         return 1;
30     }
31
32     // Initialize host arrays
33     for (int i = 0; i < N; ++i) {
34         h_A[i] = i;
35         h_B[i] = i * 2;
36     }
37
38     // Allocate device (GPU) memory
39     int *d_A = NULL;
40     int *d_B = NULL;
41     int *d_C = NULL;
42
43     cudaMalloc((void **)&d_A, size);
44     cudaMalloc((void **)&d_B, size);
45     cudaMalloc((void **)&d_C, size);
46
47     // Create CUDA events for timing
48     cudaEvent_t start, stop;
49     cudaEventCreate(&start);
50     cudaEventCreate(&stop);
51 }
```

```

51
52     // Copy data from host to device
53     cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
54     cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
55
56     // Define grid and block dimensions
57     // We use blocks of 256 threads
58     int threadsPerBlock = 256;
59     // Calculate number of blocks needed to cover all elements
60     int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
61
62     // Start timing
63     cudaEventRecord(start);
64
65     // Launch the CUDA kernel
66     // The <<<blocksPerGrid, threadsPerBlock>>> syntax configures the execution grid
67     vectorAdd<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, N);
68
69     // Stop timing
70     cudaEventRecord(stop);
71
72     // Copy result back to host
73     cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
74
75     // Wait for the stop event to complete to get accurate timing
76     cudaEventSynchronize(stop);
77     float milliseconds = 0;
78     cudaEventElapsedTime(&milliseconds, start, stop);
79
80     // Verify results
81     printf("CUDA Vector Addition\n");
82     printf("Array size: %d\n", N);
83     printf("C[0] = %d (Expected: %d)\n", h_C[0], 0 + 0);
84     printf("C[%d] = %d (Expected: %d)\n", N-1, h_C[N-1], (N-1) + (N-1)*2);
85     printf("Execution time: %f seconds\n", milliseconds / 1000.0f);
86
87     // Clean up
88     cudaEventDestroy(start);
89     cudaEventDestroy(stop);
90
91     // Free device memory
92     cudaFree(d_A);
93     cudaFree(d_B);
94     cudaFree(d_C);
95
96     // Free host memory
97     free(h_A);
98     free(h_B);
99     free(h_C);
100
101     return 0;
102 }
103

```

```
▶ !nvcc vector_add_cuda.cu -o vector_add_cuda  
!./vector_add_cuda
```

```
... nvcc warning : Support for offline compilation for architectures  
CUDA Vector Addition  
Array size: 10000000  
C[0] = 0 (Expected: 0)  
C[9999999] = 29999997 (Expected: 29999997)  
Execution time: 0.089917 seconds
```

2.2 After Optimization

1. OpenMP

```
optimized programs > openmp > c vector_add_openmp.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <omp.h>
4  #include <sys/time.h>
5
6  #define N 10000000 // Array size
7
8  // Helper function to get current time in seconds
9  double get_time() {
10     struct timeval tv;
11     gettimeofday(&tv, NULL);
12     return tv.tv_sec + tv.tv_usec * 1e-6;
13 }
14
15 int main() {
16     int *A, *B, *C;
17     double start_time, end_time;
18
19     // Allocate memory for the arrays
20     A = (int*)malloc(N * sizeof(int));
21     B = (int*)malloc(N * sizeof(int));
22     C = (int*)malloc(N * sizeof(int));
23
24     if (A == NULL || B == NULL || C == NULL) {
25         printf("Memory allocation failed!\n");
26         return 1;
27     }
28
29     // Initialize arrays
30     for (int i = 0; i < N; i++) {
31         A[i] = i;
32         B[i] = i * 2;
33     }
34
35     start_time = get_time();
36
37     // Perform vector addition using OpenMP with SIMD vectorization and static scheduling
38     #pragma omp parallel for simd schedule(static)
39     for (int i = 0; i < N; i++) {
40         C[i] = A[i] + B[i];
41     }
42
43     end_time = get_time();
44
45     // Verify results (check first and last elements)
46     printf("OpenMP Vector Addition\n");
47     printf("Array size: %d\n", N);
48     printf("C[0] = %d (Expected: %d)\n", C[0], 0 + 0);
49     printf("C[%d] = %d (Expected: %d)\n", N-1, C[N-1], (N-1) + (N-1)*2);
50     printf("Execution time: %f seconds\n", end_time - start_time);
51
52     // Free memory
53     free(A);
54     free(B);
55     free(C);
56
57     return 0;
58 }
59
```

```

heshan@heshan-Swift3:~/Desktop/PC Assignment 2 Optimized/PC---Assignment-2/optimized programs$ cd openmp/
heshan@heshan-Swift3:~/Desktop/PC Assignment 2 Optimized/PC---Assignment-2/optimized programs/openmp$ gcc -fopenmp vector_add_openmp.c -o vector_add_openmp
heshan@heshan-Swift3:~/Desktop/PC Assignment 2 Optimized/PC---Assignment-2/optimized programs/openmp$ ./vector_add_openmp
OpenMP Vector Addition
Array size: 10000000
C[0] = 0 (Expected: 0)
C[9999999] = 299999997 (Expected: 299999997)
Execution time: 0.007603 seconds
heshan@heshan-Swift3:~/Desktop/PC Assignment 2 Optimized/PC---Assignment-2/optimized programs/openmp$ 

```

SIMD Vectorization & Static Scheduling

- **File Modified:** Optimized programs/openmp/vector_add_openmp.c
- **What changed:** We updated the OpenMP pragma from `#pragma omp parallel for` to `#pragma omp parallel for simd schedule(static)`.

How it affects the result:

- **simd:** Allows the CPU to use specialized vector instructions (like AVX) to process multiple array elements simultaneously within a single clock cycle, on top of the existing multi-threading.
- **schedule(static):** Tells OpenMP to divide the loop iterations evenly among threads at compile time. Since adding two numbers always takes the exact same amount of time, this eliminates the runtime overhead of dynamically calculating who gets which chunk of the array.
- **Expected Result:** The output will be identical, but the execution time in seconds should drop noticeably, especially on CPUs with advanced vector units.

Execution Time and Performance comparison

Before Optimizing (Sec)	After Optimizing (Sec)
0.008886	0.007603

2. MPI

```
optimized programs > mpi > c vector_add_mpi.c
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  #define N 10000000 // Total array size
6
7  int main(int argc, char** argv) {
8      int rank, size;
9      int *A = NULL, *B = NULL, *C = NULL;
10     int *local_A, *local_B, *local_C;
11     double start_time, end_time;
12
13     // Initialize MPI environment
14     MPI_Init(&argc, &argv);
15     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
16     MPI_Comm_size(MPI_COMM_WORLD, &size);
17
18     // Calculate elements per process and displacements for Scatterv/Gatherv
19     int *sendcounts = (int*)malloc(size * sizeof(int));
20     int *displs = (int*)malloc(size * sizeof(int));
21
22     int sum = 0;
23     int rem = N % size; // elements remaining after division
24     for (int i = 0; i < size; i++) {
25         sendcounts[i] = N / size + (i < rem ? 1 : 0);
26         displs[i] = sum;
27         sum += sendcounts[i];
28     }
29
30     int local_N = sendcounts[rank];
31
32     // Root process initializes the full arrays
33     if (rank == 0) {
34         A = (int*)malloc(N * sizeof(int));
35         B = (int*)malloc(N * sizeof(int));
36         C = (int*)malloc(N * sizeof(int));
37
38         if (A == NULL || B == NULL || C == NULL) {
39             printf("Memory allocation failed!\n");
40             MPI_Abort(MPI_COMM_WORLD, 1);
41         }
42
43         for (int i = 0; i < N; i++) {
44             A[i] = i;
45             B[i] = i * 2;
46         }
47     }
48 }
```

```

49 // Allocate memory for local arrays on all processes
50 local_A = (int*)malloc(local_N * sizeof(int));
51 local_B = (int*)malloc(local_N * sizeof(int));
52 local_C = (int*)malloc(local_N * sizeof(int));
53
54 // Synchronize processes before starting the timer
55 MPI_Barrier(MPI_COMM_WORLD);
56 if (rank == 0) start_time = MPI_Wtime();
57
58 // Scatter the arrays A and B from root to all processes using Scatterv
59 MPI_Scatterv(A, sendcounts, displs, MPI_INT, local_A, local_N, MPI_INT, 0, MPI_COMM_WORLD);
60 MPI_Scatterv(B, sendcounts, displs, MPI_INT, local_B, local_N, MPI_INT, 0, MPI_COMM_WORLD);
61
62 // Perform local vector addition
63 for (int i = 0; i < local_N; i++) {
64     local_C[i] = local_A[i] + local_B[i];
65 }
66
67 // Gather the local results back to the root process array C using Gatherv
68 MPI_Gatherv(local_C, local_N, MPI_INT, C, sendcounts, displs, MPI_INT, 0, MPI_COMM_WORLD);
69
70 // Synchronize again to accurately measure time
71 MPI_Barrier(MPI_COMM_WORLD);
72
73 if (rank == 0) {
74     end_time = MPI_Wtime();
75     printf("MPI Vector Addition\n");
76     printf("Array size: %d, Processes: %d\n", N, size);
77     printf("C[0] = %d (Expected: %d)\n", C[0], 0 + 0);
78     printf("C[%d] = %d (Expected: %d)\n", N-1, C[N-1], (N-1) + (N-1)*2);
79     printf("Execution time: %f seconds\n", end_time - start_time);
80
81     free(A);
82     free(B);
83     free(C);
84 }
85
86 free(local_A);
87 free(local_B);
88 free(local_C);
89 free(sendcounts);
90 free(displs);
91
92 // Finalize MPI environment
93 MPI_Finalize();
94 return 0;
95 }

```

```

C[9999999] = 29999997 (Expected: 29999997)
heshan@heshan-Swift3:~/Desktop/PC Assignment 2 Optimized/PC---Assignment-2/optimized programs/mpi$ mpirun
-np 4 ./vector_add_mpi
MPI Vector Addition
Array size: 10000000, Processes: 4
C[0] = 0 (Expected: 0)
C[9999999] = 29999997 (Expected: 29999997)
Execution time: 0.082525 seconds
heshan@heshan-Swift3:~/Desktop/PC Assignment 2 Optimized/PC---Assignment-2/optimized programs/mpi$

```

Arbitrary Array Sizes (Scatterv/Gatherv)

- **File Modified:** optimized programs/mpi/vector_add_mpi.c
- **What changed:** We replaced MPI_Scatter and MPI_Gather with their variable-sized counterparts, MPI_Scatterv and MPI_Gatherv. We also added logic to compute sendcounts and displs arrays to handle division remainders.

How it affects the result:

- **Variable Data Sizes:** Before, if you set $N = 10$ but ran with 3 processes (-n 3), the program would crash and throw an error because 10 is not divisible by 3.
- **Load Balancing:** The new code calculates exactly how many elements each process should take (e.g., Process 0 takes 4, Process 1 takes 3, Process 2 takes 3) and distributes the data perfectly without crashing.
- **Expected Result:** The program is now much more robust. You can change N to any odd number or use any random number of processes without breaking the calculation, while maintaining optimal data distribution over the network.

Execution Time and Performance comparison

Before Optimizing (Sec)	After Optimizing (Sec)
0.105924	0.082525

3. CUDA

```
optimized programs > cuda > vector_add_cuda.cu
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <cuda_runtime.h>
4
5  #define N 10000000 // Array size
6
7  // CUDA Kernel function for vector addition
8  // __global__ indicates this runs on the GPU and is called from the host (CPU)
9  __global__ void vectorAdd(const int *__restrict__ A, const int *__restrict__ B, int *__restrict__ C, int numElements) {
10     // Grid-stride loop allows handling arrays larger than the grid
11     // and can improve performance by reducing kernel launch overhead
12     for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < numElements; i += blockDim.x * gridDim.x) {
13         C[i] = A[i] + B[i];
14     }
15 }
16
17 int main(void) {
18     size_t size = N * sizeof(int);
19
20     // Allocate host (CPU) memory
21     int *h_A = (int *)malloc(size);
22     int *h_B = (int *)malloc(size);
23     int *h_C = (int *)malloc(size);
24
25     if (h_A == NULL || h_B == NULL || h_C == NULL) {
26         printf("Failed to allocate host vectors!\n");
27         return 1;
28     }
29
30     // Initialize host arrays
31     for (int i = 0; i < N; ++i) {
32         h_A[i] = i;
33         h_B[i] = i * 2;
34     }
35
36     // Allocate device (GPU) memory
37     int *d_A = NULL;
38     int *d_B = NULL;
39     int *d_C = NULL;
40
41     cudaMalloc((void **)&d_A, size);
42     cudaMalloc((void **)&d_B, size);
43     cudaMalloc((void **)&d_C, size);
44
45     // Create CUDA events for timing
46     cudaEvent_t start, stop;
47     cudaEventCreate(&start);
48     cudaEventCreate(&stop);
49 }
```

```

49
50 // Copy data from host to device
51 cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
52 cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);
53
54 // Define grid and block dimensions
55 // We use blocks of 256 threads
56 int threadsPerBlock = 256;
57 // Calculate number of blocks needed to cover all elements
58 int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;
59
60 // Start timing
61 cudaEventRecord(start);
62
63 // Launch the CUDA kernel
64 // The <<<blocksPerGrid, threadsPerBlock>>> syntax configures the execution grid
65 vectorAdd<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);
66
67 // Stop timing
68 cudaEventRecord(stop);
69
70 // Copy result back to host
71 cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);
72
73 // Wait for the stop event to complete to get accurate timing
74 cudaEventSynchronize(stop);
75 float milliseconds = 0;
76 cudaEventElapsedTime(&milliseconds, start, stop);
77
78 // Verify results
79 printf("CUDA Vector Addition\n");
80 printf("Array size: %d\n", N);
81 printf("C[0] = %d (Expected: %d)\n", h_C[0], 0 + 0);
82 printf("C[%d] = %d (Expected: %d)\n", N-1, h_C[N-1], (N-1) + (N-1)*2);
83 printf("Execution time: %f seconds\n", milliseconds / 1000.0f);
84
85 // Clean up
86 cudaEventDestroy(start);
87 cudaEventDestroy(stop);
88
89 // Free device memory
90 cudaFree(d_A);
91 cudaFree(d_B);
92 cudaFree(d_C);
93
94 // Free host memory
95 free(h_A);
96 free(h_B);
97 free(h_C);
98
99 return 0;
100 }
101

```

```
!nvcc vector_add_cuda.cu -o vector_add_cuda
!./vector_add_cuda

... nvcc warning : Support for offline compilation for architecture
CUDA Vector Addition
Array size: 10000000
C[0] = 0 (Expected: 0)
C[9999999] = 29999997 (Expected: 29999997)
Execution time: 0.088439 seconds
```

Grid-Stride Loops & Memory Hints

- **File Modified:** optimized programs/cuda/vector_add_cuda.cu
- **What changed:**
 - Added `__restrict__` to the pointer arguments in the kernel `__global__ void vectorAdd(const int * __restrict__ A...`
 - Changed the basic index calculation (`int i = blockDim.x * blockIdx.x + threadIdx.x; if(i < n)`) to a for loop with a "grid-stride" (`for (int i = ...; i < numElements; i += blockDim.x * gridDim.x)`).

How it affects the result:

- **`__restrict__`:** This is a hint to the NVCC compiler guaranteeing that the memory arrays A, B, and C never overlap. The compiler can then aggressively cache reads and bypass redundant memory checks, heavily improving memory bandwidth (the primary bottleneck for vector addition).
- **Grid-Stride Loop:** Previously, if your array size exceeded the maximum number of threads your GPU could spawn at once, the program would fail or silently skip elements. With a grid-stride loop, each thread processes one element, jumps forward by the total grid size, and processes another element until the array is finished.
- **Expected Result:** You now have a highly resilient, enterprise-grade kernel. Memory fetches will be optimally pipelined by the compiler.

Execution Time and Performance comparison

Before Optimizing (Sec)	After Optimizing (Sec)
0.089917	0.088439